

DB에 넣을 것

admins

- 관리자 포털 로그인용 계정
- 비밀번호는 **해시저장**

email_accounts

- 이메일 계정 정보
- 이메일 주소, 표시명, 부서, 활성화 여부
- 비밀번호가 필요하면 **평문 말고 해시저장**

audit_logs

- 누가 로그인했고 어떤 계정을 만들었는지 기록

WSL2에 MariaDB 설치

설치

sudo apt update

sudo apt install -y mariadb-server

```
apogatto@DESKTOP-LEHJIUV:~/SME_ATTACK_LAB$ sudo apt update
[sudo] password for apogatto:
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Get:5 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1875 kB]
Ign:5 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1663 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1875 kB]
Fetched 3662 kB in 37s (99.1 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
96 packages can be upgraded. Run 'apt list --upgradable' to see them.
apogatto@DESKTOP-LEHJIUV:~/SME_ATTACK_LAB$ sudo apt install -y mariadb-server
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  galera-4 libcgi-fast-perl libcgi-pm-perl libclone-perl libconfig-inifiles-perl libdbd-mysql-perl libdbi-perl
  libencode-locale-perl libfcgi-bin libfcgi-perl libfcgi0t64 libhtml-parser-perl libhtml-tagset-perl
  libhtml-template-perl libhttp-date-perl libhttp-message-perl libio-html-perl liblwp-mediatypes-perl libmariadb3
  libmysqlclient21 libncurses6 libnuma1 libsnappy1v5 libterm-readkey-perl libtimedate-perl liburi-perl liburing2
  libwrap0 mariadb-client mariadb-client-core mariadb-common mariadb-plugin-provider-bzip2 mariadb-plugin-provider-lz4
  mariadb-plugin-provider-lzma mariadb-plugin-provider-lzo mariadb-plugin-provider-snappy mariadb-server-core
  mysql-common pv socat
Suggested packages:
  libmldbm-perl libnet-daemon-perl libsql-statement-perl libdata-dump-perl libipc-sharedcache-perl
```

실행

WSL2에서 systemd가 켜져 있으면:

sudo systemctl enable mariadb

sudo systemctl start mariadb

```
apogatto@DESKTOP-LEHJIUV:~/SME_ATTACK_LAB$ sudo systemctl enable mariadb
Synchronizing state of mariadb.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable mariadb
apogatto@DESKTOP-LEHJIUV:~/SME_ATTACK_LAB$ sudo systemctl start mariadb
```

결과값 없는게 정상.

db 초기 보안 설정

sudo mysql_secure_installation

```
apogatto@DESKTOP-LEHJIUV:~/SME_ATTACK_LAB$ sudo mysql_secure_installation
```

NOTE: RUNNING ALL PARTS OF THIS SCRIPT IS RECOMMENDED FOR ALL MariaDB
SERVERS IN PRODUCTION USE! PLEASE READ EACH STEP CAREFULLY!

In order to log into MariaDB to secure it, we'll need the current
password for the root user. If you've just installed MariaDB, and
haven't set the root password yet, you should just press enter here.

Enter current password for root (enter for none):

OK, successfully used password, moving on...

Setting the root password or using the unix_socket ensures that nobody
can log into the MariaDB root user without the proper authorisation.

You already have your root account protected, so you can safely answer 'n'.

Switch to unix_socket authentication [Y/n] y

Enabled successfully!

Reloading privilege tables..

... Success!

현재 root 비밀번호: 엔터

unix_socket 인증 사용: Y 또는 N 둘 다 가능

root 비밀번호 변경: Y

익명 사용자 제거: Y

원격 root 로그인 차단: Y

test DB 제거: N

권한 테이블 재적용: Y

DB 시작 명령어

sudo mariadb -u root

```
apogatto@DESKTOP-LEHJIUV:~/SME_ATTACK_LAB$ sudo mariadb -u root -p
```

Enter password:

Welcome to the MariaDB monitor. Commands end with ; or \g.

Your MariaDB connection id is 41

Server version: 10.11.14-MariaDB-0ubuntu0.24.04.1 Ubuntu 24.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

DB SQL (그대로 복붙)

```
-----  
CREATE DATABASE admin_portal CHARACTER SET utf8mb4 COLLATE  
utf8mb4_unicode_ci;
```

```
CREATE USER 'portal_user'@'%' IDENTIFIED BY 'StrongPortalPass!123';  
GRANT ALL PRIVILEGES ON admin_portal.*TO 'portal_user'@'%';  
FLUSH PRIVILEGES;
```

```
USE admin_portal;
```

```
CREATE TABLE admins (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  username VARCHAR(50) NOT NULL UNIQUE,  
  password_hash VARCHAR(255) NOT NULL,  
  full_name VARCHAR(100) NOT NULL,  
  role ENUM('superadmin', 'admin', 'viewer') NOT NULL DEFAULT 'admin',  
  is_active TINYINT(1) NOT NULL DEFAULT 1,  
  last_login_at DATETIME NULL,  
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE  
CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE email_accounts (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  email_address VARCHAR(255) NOT NULL UNIQUE,  
  login_id VARCHAR(100) NOT NULL UNIQUE,  
  display_name VARCHAR(100) NOT NULL,  
  department VARCHAR(100) NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  mail_password VARCHAR(255) NULL,  
  mailbox_quota_mb INT NOT NULL DEFAULT 1024,  
  is_active TINYINT(1) NOT NULL DEFAULT 1,  
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE  
CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE audit_logs (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  admin_id INT NULL,  
  action VARCHAR(100) NOT NULL,  
  target_type VARCHAR(50) NULL,  
  target_id INT NULL,  
  ip_address VARCHAR(45) NULL,  
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT fk_audit_admin  
    FOREIGN KEY (admin_id) REFERENCES admins(id)  
    ON DELETE SET NULL  
);  
-----
```

이후 php로 해시 생성하기 위해 php를 설치

sudo apt install php -y

```
apogatto@DESKTOP-LEHJ1UV:~/SME_ATTACK_LAB$ sudo apt install php -y
[sudo] password for apogatto:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  apache2 apache2-bin apache2-data apache2-utils libapache2-mod-php8.3 libapr1t64 libaprutil1-dbd-sqlite3
  libaprutil1-ldap libaprutil1t64 liblua5.4-0 php-common php8.3 php8.3-cli php8.3-common php8.3-opcache
  php8.3-readline ssl-cert
Suggested packages:
  apache2-doc apache2-suexec-pristine | apache2-suexec-custom www-browser ufw php-pear
The following NEW packages will be installed:
  apache2 apache2-bin apache2-data apache2-utils libapache2-mod-php8.3 libapr1t64 libaprutil1-dbd-sqlite3
  libaprutil1-ldap libaprutil1t64 liblua5.4-0 php php-common php8.3 php8.3-cli php8.3-common php8.3-opcache
```

설치 후 기존 관리자 비밀번호를 입력 시 해시 값 생성

```
apogatto@DESKTOP-LEHJ1UV:~/SME_ATTACK_LAB$ php -r "echo password_hash('qhdksdlchlrx', PASSWORD_DEFAULT) . PHP_EOL;"
$2y$10$LBHy6xbLHwNmB/5ruJaRV.hR4Rl5Aym7TIR..HCUt591Db2yf87Je
```

이후 DB 다시 접속 후 계정 등록

```
INSERT INTO admins (username, password_hash, full_name, role, is_active)
VALUES(
  'admin',
  '$2y$10$LBHy6xbLHwNmB/5ruJaRV.hR4Rl5Aym7TIR..HCUt591Db2yf87Je',
  'System Admin',
  'superadmin',
  1
);
```

삽입 내용 확인

select * from admins;

```
MariaDB [admin_portal]> select * from admins;
+----+-----+-----+-----+-----+-----+-----+-----+
| id | username | password_hash | full_name | role | is_active | last_login_at | created_at | updated_at |
+----+-----+-----+-----+-----+-----+-----+-----+
| 1 | admin | $2y$10$LBHy6xbLHwNmB/5ruJaRV.hR4Rl5Aym7TIR..HCUt591Db2yf87Je | System Admin | superadmin | 1 | NULL | 2026-04-04 18:24:38 | 2026-04-04 18:30:27 |
+----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.000 sec)
```

php와 DB를 연결하기 위한 서비스 설치

sudo apt install php-mysql -y

sudo service apache2 restart

여기서 잠깐

우린 nginx를 80포트로 쓰고있기에 apache2와 포트가 겹침
그래서 apache2를 8080으로 옮기는 것을 추천

sudo nano /etc/apache2/ports.conf

Listen 80을 8080으로 수정

```
# If you just change the port or add more ports here, you will likely also
# have to change the VirtualHost statement in
# /etc/apache2/sites-enabled/000-default.conf

Listen 8080
```

사이트 설정도 같이 수정

sudo nano /etc/apache2/sites-available/000-default.conf

```
<VirtualHost *:8080>
# The ServerName directive sets the request scheme, hostname and port that
# the server uses to identify itself. This is used when creating
# redirection URLs. In the context of virtual hosts, the ServerName
# specifies what hostname must appear in the request's Host: header to
# match this virtual host. For the default virtual host (this file) this
# value is not decisive as it is used as a last resort host regardless.
# However, you must set it for any further virtual host explicitly.
#ServerName www.example.com

ServerAdmin webmaster@localhost
DocumentRoot /var/www/html
```

맨 위 virtualhost를 8080으로

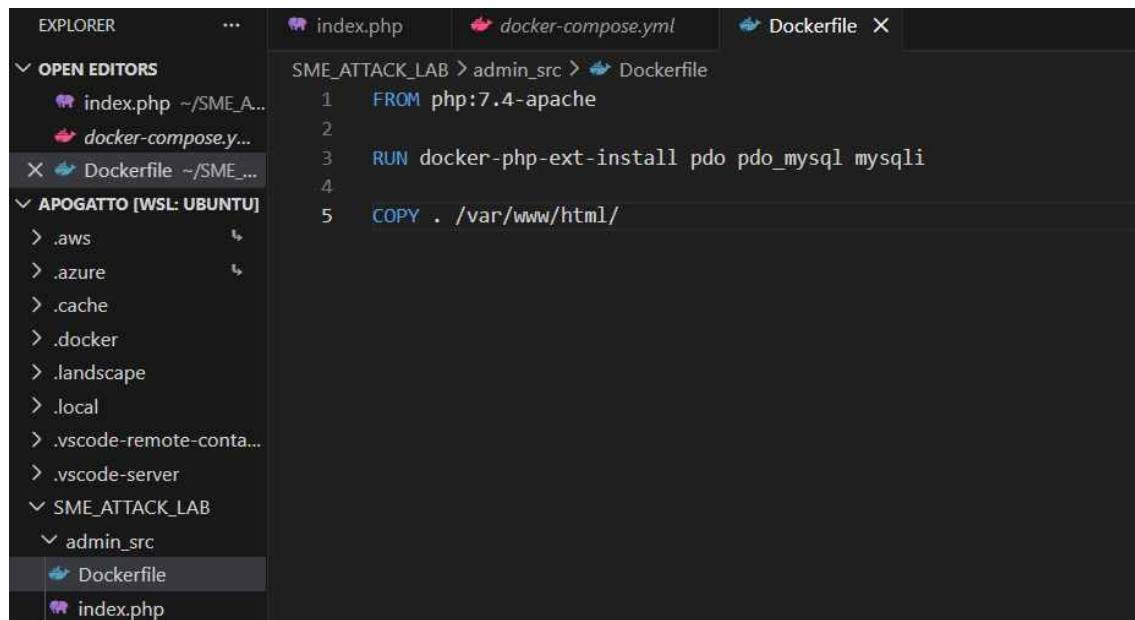
파일 수정은 제가 작성한거 올리긴 할텐데 어디가 바뀌었는
지 표시만 하겠습니다.

수정되는 파일

- Dockerfile (추가됨)
- docker-compose.yml (이미지 사용에서 내부 파일 빌드로 수정)
- index.php (계정 로그인 DB연동 추가)

이제 admin_antry를 다시 빌드하기 위한 Dockerfile을 추가

```
FROM php:7.4-apache
RUN docker-php-ext-install pdo pdo_mysql mysqli
COPY . /var/www/html/
```



docker-compose.yml 수정

```
services:
  # --- 1. 관리자 포털 (PHP로 변경) ---
  admin_entry:
    build: ./admin_src
    container_name: admin_entry
    ports:
      - "8003:80"
    volumes:
      - ./admin_src:/var/www/html
    extra_hosts:
      - "host.docker.internal:host-gateway"
    environment:
      DB_HOST: host.docker.internal
      DB_NAME: admin_portal
      DB_USER: portal_user
      DB_PASS: StrongPortalPass!123
    networks:
      sme_internal_network:
        ipv4_address: 192.168.20.40
```

시작부분 (session_start(); 바로 아래쯤)

```
<?php
session_start();

// === DB 설정 ===
$DB_HOST = getenv('DB_HOST') ?: 'host.docker.internal'; // 컨테이너면
host.docker.internal, WSL 내부 PHP면 127.0.0.1
$DB_NAME = getenv('DB_NAME') ?: 'admin_portal';
$DB_USER = getenv('DB_USER') ?: 'portal_user';
$DB_PASS = getenv('DB_PASS') ?: 'StrongPortalPass!123';

try {
    $pdo = new PDO(
        "mysql:host={$DB_HOST};dbname={$DB_NAME};charset=utf8mb4",
        $DB_USER,
        $DB_PASS,
        [
```

```

<?php
session_start();

// === DB 설정 ===
$DB_HOST = getenv('DB_HOST') ?: 'host.docker.internal'; // 컨테이너면 host.docker.internal, WSL 내부 PHP면 1
$DB_NAME = getenv('DB_NAME') ?: 'admin_portal';
$DB_USER = getenv('DB_USER') ?: 'portal_user';
$DB_PASS = getenv('DB_PASS') ?: 'StrongPortalPass!123';

try {
    $pdo = new PDO(
        "mysql:host={$DB_HOST};dbname={$DB_NAME};charset=utf8mb4",
        $DB_USER,
        $DB_PASS,
        [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES => false,
        ]
    );
} catch (PDOException $e) {
    die('DB 연결 실패: ' . $e->getMessage());
}

```

```

        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES => false,
    ]
);
} catch (PDOException $e) {
    die('DB 연결 실패: ' . $e->getMessage());
}
?>

```

login 기능

```
if ($action === 'login') {
    $data = json_decode(file_get_contents('php://input'), true) ?: [];
    $user = trim($data['user'] ?? '');
    $pw = $data['pw'] ?? '';

    if ($user === '' || $pw === '') {
        echo json_encode(['ok' => false, 'error' => '아이디와 비밀번호를 입력하세요.']);
        exit;
    }

    $stmt = $pdo->prepare("
        SELECT id, username, password_hash, full_name, role, is_active
        FROM admins
        WHERE username = ?
        LIMIT 1
    ");
    $stmt->execute([$user]);
    $admin = $stmt->fetch();

    if ($admin && (int)$admin['is_active'] === 1 && password_verify($pw, $admin['password_hash'])) {
        session_regenerate_id(true);

        $_SESSION['admin'] = $admin['username']; // 기존 화면 호환용
        $_SESSION['admin_id'] = (int)$admin['id'];
        $_SESSION['full_name'] = $admin['full_name'];
        $_SESSION['role'] = $admin['role'];

        $pdo->prepare("UPDATE admins SET last_login_at = NOW() WHERE id = ?")
            ->execute([$admin['id']]);

        $pdo->prepare("
            INSERT INTO audit_logs (admin_id, action, target_type, ip_address)
            VALUES (?, 'LOGIN_SUCCESS', 'admin', ?)
        ")>execute([
```

```
if ($action === 'login') {
    $data = json_decode(file_get_contents('php://input'), true) ?: [];
    $user = trim($data['user'] ?? '');
    $pw = $data['pw'] ?? '';

    if ($user === '' || $pw === '') {
        echo json_encode(['ok' => false, 'error' => '아이디와 비밀번호를 입력하세요.']);
        exit;
    }

    $stmt = $pdo->prepare("
        SELECT id, username, password_hash, full_name, role, is_active
        FROM admins
        WHERE username = ?
        LIMIT 1
    ");
    $stmt->execute([$user]);
    $admin = $stmt->fetch();

    if ($admin && (int)$admin['is_active'] === 1 && password_verify($pw,
    $admin['password_hash'])) {
        session_regenerate_id(true);
```

```

$_SESSION['admin'] = $admin['username'];    // 기존 화면 호환용
$_SESSION['admin_id'] = (int)$admin['id'];
$_SESSION['full_name'] = $admin['full_name'];
$_SESSION['role'] = $admin['role'];

$pdo->prepare("UPDATE admins SET last_login_at = NOW() WHERE id = ?")
->execute([$admin['id']]);

$pdo->prepare("
    INSERT INTO audit_logs (admin_id, action, target_type, ip_address)
    VALUES (?, 'LOGIN_SUCCESS', 'admin', ?)
")->execute([
    $admin['id'],
    $_SERVER['REMOTE_ADDR'] ?? null
]);

echo json_encode([
    'ok' => true,
    'user' => $admin['username'],
    'full_name' => $admin['full_name'],
    'role' => $admin['role']
]);
exit;
}

$pdo->prepare("
    INSERT INTO audit_logs (admin_id, action, target_type, ip_address)
    VALUES (NULL, 'LOGIN_FAIL', 'admin', ?)
")->execute([
    $_SERVER['REMOTE_ADDR'] ?? null
]);

echo json_encode(['ok' => false, 'error' => '인증 실패']);
exit;
}

```

로그아웃 기능

```
if ($action === 'logout') {
    $_SESSION = [];
    if (ini_get("session.use_cookies")) {
        $params = session_get_cookie_params();
        setcookie(session_name(), '', time() - 42000,
            $params["path"], $params["domain"],
            $params["secure"], $params["httponly"]
        );
    }
    session_destroy();
    echo json_encode(['ok' => true]);
    exit;
}
```

```
if ($action === 'logout') {
    $_SESSION = [];
    if (ini_get("session.use_cookies")) {
        $params = session_get_cookie_params();
        setcookie(session_name(), "", time() - 42000,
            $params["path"], $params["domain"],
            $params["secure"], $params["httponly"]
        );
    }
    session_destroy();
    echo json_encode(['ok' => true]);
    exit;
}
```

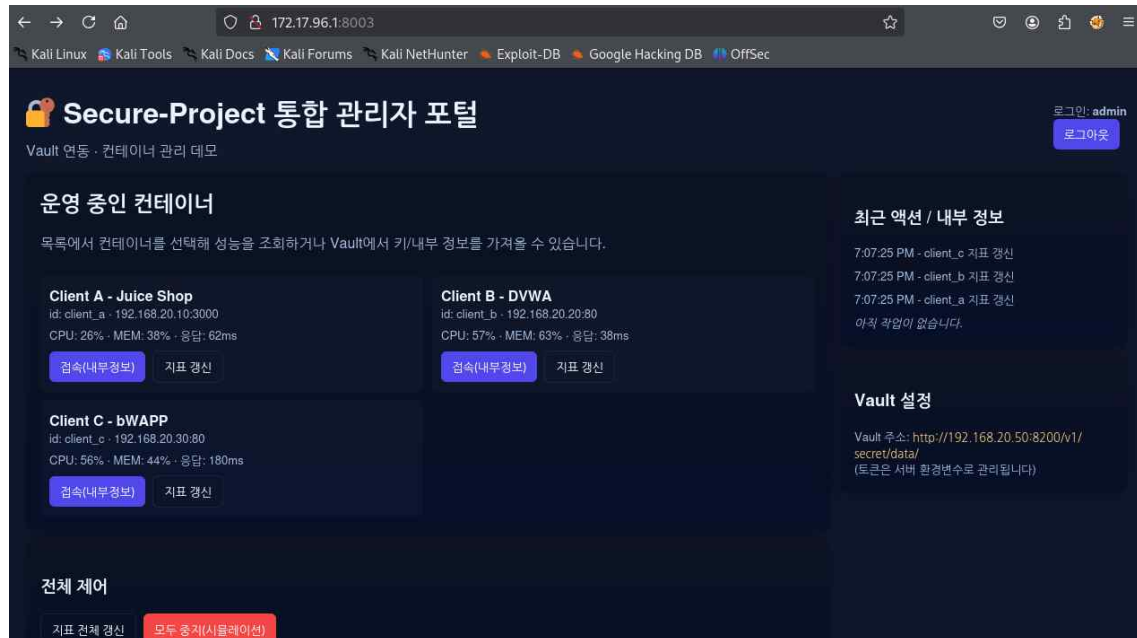
기존 admin and try 컨테이너 삭제 후 재빌드
docker compose up -d --build admin_entry

빌드 후 해당 검색이 나오면 성공

docker exec -it admin_entry php -m | grep -E 'pdo_mysql|mysqli'

```
apogatto@DESKTOP-LEHJIUV:~/SME_ATTACK_LAB$ docker exec -it admin_entry php -m | grep -E 'pdo_mysql|mysqli'
mysqli
pdo_mysql
```

그럼 이제 포털가서 로그인 해보고 정상적으로 로그인 되는지 확인



로그 확인

```
sudo mariadb -u root
select id, username, last_login_at from admins;
```

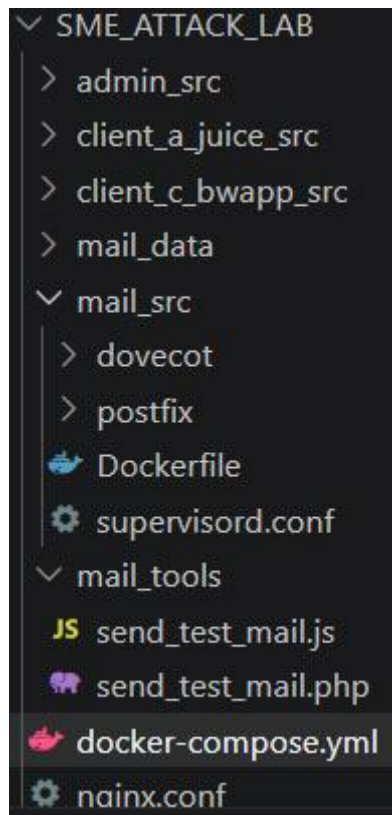
```
+-----+-----+-----+
| id | username | last_login_at |
+-----+-----+-----+
| 1 | admin | 2026-04-04 19:02:10 |
+-----+-----+-----+
1 row in set (0.000 sec)
```

```
SELECT * FROM audit_logs ORDER BY id DESC;
```

```
MariaDB [admin_portal]> SELECT * FROM audit_logs ORDER BY id DESC;
+-----+-----+-----+-----+-----+-----+
| id | admin_id | action | target_type | target_id | ip_address | created_at |
+-----+-----+-----+-----+-----+-----+
| 1 | 1 | LOGIN_SUCCESS | admin | NULL | 192.168.20.1 | 2026-04-04 19:02:10 |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.000 sec)
```

mail server 설정

[mail_src] 폴더 내 구성도



****작성된 파일이 많아 한번에 업로드된 파일을 적용할 것을 권장****

****주의할 점****

`docker exec -it mail_server sh -lc "getent hosts host.docker.internal"`

```
apogatto@DESKTOP-LEHJIUV:~/SME_ATTACK_LAB$ docker exec -it mail_server sh -lc "getent hosts host.docker.internal"
192.168.65.254 host.docker.internal
```

해당 주소와 다르게 표시될 시

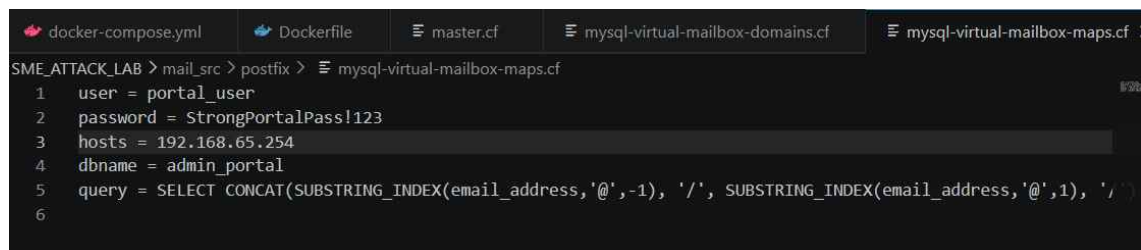
mysql-virtual-mailbox-domains.cf

mysql-virtual-mailbox-maps.cf

dovecot-sql.conf.ext

3개의 파일 내부에서 hosts 주소를 바꿔줘야함

EX) maps.cf



3개 모두 hosts 구문이 있으니 꼭 확인해서 자신의 주소로 바꿀 것.

[생성해야될 계정 /비밀번호] (나중에 포털에서 추가할 수 있음)

ceo@company.local / Mail1234

it.admin@company.local / Admin1234

support@company.local / Support1234

[mail_server 컨테이너 올리기]

docker compose down mail_server

docker compose build --no-cache mail_server

docker compose up -d mail_server

[컨테이너 Up인지, Postfix/Dovecot 에러 없는지 확인]

docker ps

docker logs -f mail_server

[시스템 확인]

docker exec -it mail_server supervisorctl status

3개가 다 RUNNING이여야함.

[포트 확인]

nc -vz 127.0.0.1 2525

nc -vz 127.0.0.1 2143

둘다 succeeded 나와야됨

[wsl에 메일함 디렉터리 생성 (SMBE_ATTACK_LAB)]

sudo mkdir -p mail_data/company.local/ceo/Maildir/{cur,new,tmp}

sudo mkdir -p mail_data/company.local/it.admin/Maildir/{cur,new,tmp}

sudo mkdir -p mail_data/company.local/support/Maildir/{cur,new,tmp}

sudo chown -R 5000:5000 mail_data

[telnet 으로 메일 보내기] (wsl 내부에서 동작하는지 체크과정)

apt install telnet -y

telnet 127.0.0.1 2525

HELO company.local

MAIL FROM:<test@company.local>

RCPT TO:<ceo@company.local>

DATA

Subject: first test

hello ceo

.

QUIT

```
apogatto@DESKTOP-LEHJIUV:~/SME_ATTACK_LAB$ telnet 127.0.0.1 2525
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
220 mail.company.local ESMTP Postfix
HELO company.local
250 mail.company.local
MAIL FROM:<test@company.local>
250 2.1.0 Ok
RCPT TO:<ceo@company.local>
250 2.1.5 Ok
DATA
354 End data with <CR><LF>.<CR><LF>
Subject: first test

hello ceo
.
250 2.0.0 Ok: queued as 7B47410AE8
QUIT
221 2.0.0 Bye
Connection closed by foreign host.
```

[telnet으로 메일 확인하기]

telnet 127.0.0.1 2143

a login ceo@company.local Mail1234

a select INBOX

a fetch 1 body[]

a logout

```
apogatto@DESKTOP-LEHJ10V:~/SME_ATTACK_LAB$ telnet 127.0.0.1 2143
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^'.
* OK [CAPABILITY IMAP4rev1 SASL-IR LOGIN-REFERRALS ID ENABLE IDLE SORT SORT=DISPLAY THREAD=REFERENCES THREAD=REFS THRE
tu) ready.
a login ceo@company.local Mail1234
a OK [CAPABILITY IMAP4rev1 SASL-IR LOGIN-REFERRALS ID ENABLE IDLE SORT SORT=DISPLAY THREAD=REFERENCES THREAD=REFS THRE
=ORDEREDSUBJECT MULTIAPPEND URL-PARTIAL CATENATE UNSELECT CHILDREN NAMESPACE UIDPLUS LIST-EXTENDED I18NLEVEL=1 CONDSTO
QRESYNC ESEARCH ESORT SEARCHRES WITHIN CONTEXT=SEARCH LIST-STATUS BINARY MOVE SNIPPET=FUZZY PREVIEW=FUZZY PREVIEW STA
S=SIZE SAVEDATE LITERAL+ NOTIFY SPECIAL-USE] Logged in
a select INBOX
* FLAGS (\Answered \Flagged \Deleted \Seen \Draft)
* OK [PERMANENTFLAGS (\Answered \Flagged \Deleted \Seen \Draft \*)] Flags permitted.
* 1 EXISTS
* 1 RECENT
* OK [UNSEEN 1] First unseen.
* OK [UIDVALIDITY 1775363171] UIDs valid
* OK [UIDNEXT 2] Predicted next UID
a OK [READ-WRITE] Select completed (0.001 + 0.000 secs).
a fetch 1 body[]
* 1 FETCH (FLAGS (\Seen \Recent) BODY[] {475}
Return-Path: <test@company.local>
Delivered-To: ceo@company.local
Received: from mail.company.local
    by mail.company.local with LMTP
    id MsbRNR7n0WmNAAAAQtQJWA
    (envelope-from <test@company.local>)
    for <ceo@company.local>; Sun, 05 Apr 2026 04:37:50 +0000
Received: from company.local (unknown [192.168.20.1])
    by mail.company.local (Postfix) with SMTP id 7B47410AE8
    for <ceo@company.local>; Sun, 5 Apr 2026 04:37:23 +0000 (UTC)
Subject: first test

hello ceo
)
a OK Fetch completed (0.001 + 0.000 secs).
```

EXISTS가 도착한 메일 수

[현재 시스템 진행 순서]

SMTP로 메일 발송 → Postfix 큐잉 → Dovecot LMTP 전달 → Maildir 저장 → IMAP 로그
인 후 조회

[ws에서 테스트 메일 보내기 (각 client의 컨테이너 내부에서 테스트용 메일 발송 명령어)]

docker exec -it client_a_owasp node /tmp/send_test_mail.js "[client_a] test" "Juice Shop
test"

docker exec -it client_b_dwva php /tmp/send_test_mail.php "[client_b] test" "DVWA test"

docker exec -it client_c_bwapp php /tmp/send_test_mail.php "[client_c] test" "bWAPP test"

현재 테스트 스크립트 동작 방식

- SMTP 호스트: SMTP_HOST (없으면 기본값 mail_server)
- SMTP 포트: SMTP_PORT (없으면 기본값 25)
- 발신자: MAIL_FROM
- 수신자: MAIL_TO
- 제목: MAIL_SUBJECT
- 본문: MAIL_BODY

[관리자 포털에서 이메일 생성 기능 추가]

이메일 계정 관리

현재 DB의 `email_accounts` 테이블을 조회하고 계정을 추가/비활성화합니다.

이메일 주소

로그인 ID

ceo@company.local

ceo

표시 이름

부서

CEO

Executive

초기 비밀번호

용량(MB)

Mail1234

1024

계정 추가

데모 채우기

계정이 추가되었습니다.

ID	Email	Login ID	표시 이름	부서	Quota	상태	등록
1	test@company.local	test	test	Executive	1024 MB	활성	비활성화

[DB에서 추가된 이메일 조회]

```
MariaDB [admin_portal]> select * from email_accounts;
```

id	email_address	login_id	display_name	department	password_hash	mailbox_quota_mb	is_active	created_at	updated_at
1	test@company.local	test	test	Executive	\$2y\$10\$tD_gp_Q3yGUXPywLbsYHMu/zEyKMYTMAWgJpyo9SE.5060x11IPVy	1024	1	2026-04-05 14:01:33	2026-04-05 14:01:33

```
1 row in set (0.000 sec)
```

```
MariaDB [admin_portal]> |
```

[client가 보낸 메일을 확인할 수 있음.]

메일 열람 (Read-only)

superadmin 전용 기능입니다. 선택한 계정으로 IMAP에 접속해 최근 메일 20개를 읽습니다.

메일 계정 선택

ceo@company.local (CEO)

메일함 불러오기

Test Mail

first test

Test Mail

/

hello from client

[로그 추가]

ws|에서

```
apogatto@DESKTOP-LEHJIUV:~/SME_ATTACK_LAB$ docker exec -it admin_entry /bin/bash
```

```
apogatto@DESKTOP-LEHJIUV:~/SME_ATTACK_LAB$ chmod 777 /var/www/html
```